S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Post Lab

Aim: To implement and demonstrate the Producer-Consumer problem (also known as the Bounded-
Buffer problem) using semaphores in C, illustrating inter-process synchronisation and mutual
exclusion in an Operating System environment

Name: Sonakshi Dhomne

Roll No: CM24049

Semester / Year: III / 2 nd

Academic Session: 2026-2027

Date of Performance:

Date of submission:

Aim: To implement and demonstrate the Producer-Consumer problem (also known as the Bounded-Buffer problem) using semaphores in C, illustrating inter-process synchronisation and mutual Exclusion in an Operating System environment.

Objectives:

- To Understand the concept of process synchronisation and the critical section problem.
- . Study the role of semaphores (binary and counting) as a synchronisation primitive.
- Implement the Producer-Consumer solution using POSIX semaphores in C.

Theory: Producer-Consumer problem is a classic synchronisation problem Operating Systems. It involves two types of processes sharing a fixed-size buffer: the Producer, which generates data and places it into the buffer, and the Consumer, which removes data from the buffer for processing. Without proper synchronisation, race conditions can occur.

Semaphores

A semaphore is an integer variable used for process synchronisation, accessed only through two

Atomic operations — wait() (P) and signal() (V). Two types are used here:

• Counting Semaphore (empty, full) — tracks the number of empty and filled slots in the buffer.

• Binary Semaphore / Mutex — ensures mutual exclusion while accessing the shared buffer.

Semaphore Definitions

Semaphore Initial Value Purpose

Empty N (buffer size) Counts empty slots in buffer

Full 0 Counts filled slots in buffer

Mutex 1 Mutual exclusion on buffer access

Department of Emerging Technologies, SBJITMR, Nagpur.

```
Code: #include <stdio.h>
   #include <stdlib.h>
   #include <pthread.h>
   #include <semaphore.h>
   #include <unistd.h>
   #define BUFFER_SIZE 5 int
   buffer[BUFFER_SIZE];
   int in = 0, out = 0; sem_t empty, full, mutex; void
   *producer(void *arg) {  int item;  for (int i = 0; i <
   10; i++) {  item = rand() % 100; // Produce item
   sem_wait(Ø );
   // Wait if buffer full  sem_wait(&mutex;); // Enter critical
   section  buffer[in] = item;  printf("Producer: Produced item %d
   at index %d\n", item, in);  in = (in + 1) % BUFFER_SIZE;
   sem_post(&mutex;); // Exit critical section  sem_post(&full;); //
   Signal item added  sleep(1);
}
    return NULL;
}
```

```
 void *consumer(void *arg) {
 int item; for (int i = 0; i < 10; i++) {
sem_wait(&full;); // Wait if buffer empty
sem_wait(&mutex;); // Enter critical section
item = buffer[out];
 printf("Consumer: Consumed item %d from index %d\n", item, out);
out = (out + 1) % BUFFER_SIZE;
 sem_post(&mutex;); // Exit critical
section  sem_post(Ø); // Signal slot freed
sleep(2);
 }
 return NULL;
}
int main() {
 pthread_t prod, cons;
sem_init(Ø, 0, BUFFER_SIZE);
 sem_init(&full;,          0,          0);
sem_init(&mutex;, 0, 1);
 pthread_create(∏, NULL, producer, NULL);
pthread_create(&cons;, NULL, consumer, NULL);
pthread_join(prod, NULL);  pthread_join(cons,
NULL);
 sem_destroy(Ø);
sem_destroy(&full;);
sem_destroy(&mutex;);
 return 0;
```

Result :

Producer: Produced item 83 at index 0

Producer: Produced item 46 at Inde” 1

Consumer: Consumed item 83 from index 0

Producer: Produced item 17 at index 2

Consumer: Consumed item 46 from index 1

Producer: Produced item 63 at index 3

Consumer: Consumed item 17 from index 2

Producer: Produced item 25 at index 4

Conclusion:

The producer and consumer operate concurrently without interfering with each other. The buffer never

Overflows (producer waits when full) and never underflows (consumer waits when empty).
Mutual exclusion is correctly maintained — no two processes access the buffer simultaneously.

- **Discussion Questions:**

1. **What is the significance of the pwd command In a Linux environment?**
2. **Explain the function of the cp command and Its common options.**
3. **How does chmod 700 affect file permissions, and what does each digit represent?**
4. **Describe the difference between head and tail commands In Linux.**
5. **What is the purpose of the grep command, and how Is it used with regular expressions?**

- **References**:

https://ubuntu.com/tutorials/command-line-for-beginners#1-overview
https://www.geeksforgeeks.org/25-basic-ubuntu-commands/

Date: / /2026 **Signature**
Course Coordinator B.Tech CSE(AIML)
Sem: 4 / 2025-26